

Inlämningsuppgift 1

Skapa & Deploya App Service på Azure

Essan Alshakerchi

.NET Cloud Developer | IT-Högskolan Göteborg | VT 2026

Genomförd: 28–29 mars 2026 | Uppdaterad: 6 maj 2026

Innehållsförteckning

1. Projektöversikt och resurser
2. Skapa Web API med Entity Framework
3. Azure App Service via Azure CLI
4. Azure SQL-databas via CLI
5. Azure Key Vault och Managed Identity
6. GitHub Actions – Automatisk deployment
7. Application Insights – Loggning
8. Säkerhet – HTTPS, IP-restriktion, Backup
9. Autoskalning
10. Azure Storage Account
11. Verifikation
12. Källförteckning

1. Projektöversikt och resurser

I denna uppgift har jag byggt ett komplett REST API och driftsatt det på Azure App Service med full CI/CD-pipeline, säkerhet och övervakning. Alla Azure-resurser har skapats via Azure CLI. Appen är live och fullt funktionell.

Resurstabell

Resurs	Värde
Projektnamn	AzureAppServiceAPI
Teknik	ASP.NET Core Web API, .NET 10
App Service	productapi-essan
App Service Plan	ASP-ProductAPI (S1)
SQL Server	sql-essan-25
SQL Databas	AzureAppServiceDB
Key Vault	kv-essan-25
Storage Account	stoproductapi1
Application Insights	insights-essan-25
GitHub Repository	essanalshakerchi-star/AzureAppServiceAPI
Region	Sweden Central
Resursgrupp	RG-Essan-Alshakerchi-10d10a-DotNetCloudDeveloper-VT-Mars-Goteborg
Subscription	SUB-Utbildning-DotNetCloudDeveloper-2026-VT-Mars-Goteborg

2. Skapa Web API med Entity Framework

Skapade ASP.NET Core Web API-projekt i Visual Studio med .NET 10. Arkitekturen följer den mönster vi arbetat med under kursen:

Entity → IRepo → Repo → IService → Service → Controller

2.1 Mappstruktur

Mapp	Beskrivning
Context/	AppDbContext.cs – kopplar EF Core mot databasen
Data/Entities/	Product.cs och Category.cs – databasmodeller
Data/DTOs/	AddProductDTO, UpdateProductDTO, AddCategoryDTO
Data/Interfaces/	IProductRepo, ICategoryRepo – gränssnitt
Data/Repos/	ProductRepo, CategoryRepo – databaskommunikation
Core/Interfaces/	IProductService, ICategoryService
Core/Services/	ProductService, CategoryService – affärslogik
Controllers/	ProductsController, CategoriesController – API-endpoints

2.2 NuGet-paket

Följande paket installerades via NuGet Package Manager (Tools → NuGet Package Manager → Manage NuGet Packages):

- ☐ Microsoft.EntityFrameworkCore.SqlServer (10.0.5) – EF Core mot SQL Server
- ☐ Microsoft.EntityFrameworkCore.Tools (10.0.5) – migrationer
- ☐ Microsoft.EntityFrameworkCore.Design (10.0.5) – design-time verktyg
- ☐ Azure.Extensions.AspNetCore.Configuration.Secrets (1.5.0) – Key Vault-integration
- ☐ Azure.Identity (1.19.0) – Managed Identity-autentisering
- ☐ Microsoft.ApplicationInsights.AspNetCore (3.0.0) – Application Insights
- ☐ Swashbuckle.AspNetCore (10.1.7) – Swagger UI

2.3 appsettings.json

appsettings.json konfigurerades med KeyVaultUrl (tom lokalt, fylls i efter Key Vault skapats):

```
{ "KeyVaultUrl": "", "AllowedHosts": "*", "Logging": { "LogLevel": { "Default": "Information" } } }
```

2.4 appsettings.Development.json

appsettings.Development.json innehåller connection string för lokal utveckling:

```
"ConnectionStrings": { "DefaultConnection":  
"Server=ESSAN\\SQLEXPRESS;Database=AzureAppServiceDB;Trusted_Connection=True;  
TrustServerCertificate=True" }
```

2.5 Program.cs – konfiguration

Program.cs konfigurerades med följande logik:

- ❑ Läs KeyVaultUrl från appsettings – om den finns (Azure-miljö) koppla till Key Vault
- ❑ Registrera controllers, Swagger och Application Insights
- ❑ Registrera DbContext med connection string från configuration
- ❑ Registrera alla Repos och Services med AddScoped (Dependency Injection)
- ❑ Kör EF Core-migrationer automatiskt vid uppstart (db.Database.Migrate())

2.6 Lokal testning

Databasen skapades lokalt via Package Manager Console i Visual Studio:

Add-Migration InitialCreate

Update-Database

Appen startades med F5 och testades i Postman. Alla endpoints returnerade 200 OK.

3. Azure App Service via Azure CLI

Alla Azure-resurser skapades via Azure CLI i Cloud Shell. Öppnade shell.azure.com och bytte till Bash-läge.

3.1 Sätt variabler (valfritt men rekommenderat)

För att slippa skriva långa resursgruppsnamn föreslår jag att sätta variabler i Bash:

RESOURCE_GROUP="RG-Essan-Alshakerchi-10d10a-DotNetCloudDeveloper-VT-Mars-Goteborg"

LOCATION="swedencentral"

APP_PLAN="ASP-ProductAPI"

APP_NAME="productapi-essan"

SUBSCRIPTION_ID="457c50ad-2cb0-4bed-9fea-fbdf6eed15bf"

3.2 Skapa App Service Plan

S1-planen (Standard) valdes för att stöda backup, autoskalning och andra produktionsfunktioner:

az appservice plan create --name ASP-ProductAPI --resource-group "RG-Essan-Alshakerchi-10d10a-DotNetCloudDeveloper-VT-Mars-Goteborg" --location swedencentral --sku S1

3.3 Skapa Web App

Web Appen skapades och kopplades automatiskt till planen:

az webapp create --name productapi-essan --resource-group "RG-Essan-Alshakerchi-10d10a-DotNetCloudDeveloper-VT-Mars-Goteborg" --plan ASP-ProductAPI --runtime "dotnet:10"

4. Azure SQL-databas via CLI

4.1 Skapa SQL Server

```
az sql server create --name sql-essan-25 --resource-group "RG-Essan-Alshakerchi-10d10a-DotNetCloudDeveloper-VT-Mars-Goteborg" --location swedencentral --admin-user sqladmin --admin-password EssanPass123!
```

4.2 Öppna brandvägsregeln för Azure-tjänster

Brandvägsregeln tillåter Azure App Service att kommunicera med SQL Server (IP 0.0.0.0 till 0.0.0.0 är en speciell regel för alla interna Azure-tjänster):

```
az sql server firewall-rule create --resource-group "RG-Essan-Alshakerchi-10d10a-DotNetCloudDeveloper-VT-Mars-Goteborg" --server sql-essan-25 --name AllowAzureServices --start-ip-address 0.0.0.0 --end-ip-address 0.0.0.0
```

4.3 Skapa databas

```
az sql db create --resource-group "RG-Essan-Alshakerchi-10d10a-DotNetCloudDeveloper-VT-Mars-Goteborg" --server sql-essan-25 --name AzureAppServiceDB --service-objective Basic
```

5. Azure Key Vault och Managed Identity

Key Vault lagrar connection string säkert. Appen hämtar det via Managed Identity utan lösenord i koden.

5.1 Varför Key Vault?

Om connection string lagras direkt i koden eller appsettings.json kan den hamna i publika repositories. Key Vault löser detta genom att kryptera all data, kräva autentisering, logga åtkomster och möjliggöra rotation av lösenord.

5.2 Skapa Key Vault

```
az keyvault create --name kv-essan-25 --resource-group "RG-Essan-Alshakerchi-10d10a-DotNetCloudDeveloper-VT-Mars-Goteborg" --location swedencentral --enable-rbac-authorization true
```

RBAC (Role-Based Access Control) valdes som permission model – detta är det rekommenderade sättet enligt kursen.

5.3 Ge mig själv rättighet att skapa secrets

Via Azure Portal: kv-essan-25 → Access Control (IAM) → Add role assignment → Key Vault Secrets Officer → Mitt konto → Review + Assign

5.4 Lägga in connection string som secret

```
az keyvault secret set --vault-name kv-essan-25 --name "ConnectionStrings--DefaultConnection" --value "Server=tcp:sql-essan-25.database.windows.net,1433;Database=AzureAppServiceDB;UserID=sqladmin;Password=EssanPass123!;Encrypt=True;TrustServerCertificate=False;Connection Timeout=30;"
```

Secret-namnet ConnectionStrings--DefaultConnection matchar hur ASP.NET Core läser configuration (dubbelt bindestreck ersätter kolon).

5.5 Aktivera Managed Identity

```
az webapp identity assign --name productapi-essan --resource-group "RG-Essan-Alshakerchi-10d10a-DotNetCloudDeveloper-VT-Mars-Goteborg"
```

Kommandot returnerar ett principalId som behövs för nästa steg.

5.6 Ge Web Appen läsrättighet till Key Vault

```
az role assignment create --role "Key Vault Secrets User" --assignee 3942b3e2-b10b-4444-97ba-7c68738e9d56 --scope "/subscriptions/457c50ad-2cb0-4bed-9fea-fbdf6eed15bf/providers/Microsoft.KeyVault/vaults/kv-essan-25"
```

(Ersätt assignee-ID med det principalId som returnerades från steg 5.5)

5.7 Sätt KeyVaultUrl App Setting

```
az webapp config appsettings set --name productapi-essan --resource-group "RG-Essan-Alshakerchi-10d10a-DotNetCloudDeveloper-VT-Mars-Goteborg" --settings KeyVaultUrl=https://kv-essan-25.vault.azure.net/
```

5.8 Program.cs integration

I Program.cs hämtas KeyVaultUrl och kopplas automatiskt. DefaultAzureCredential använder Managed Identity i Azure och Visual Studio-inloggning lokalt.

6. GitHub Actions – Automatisk deployment

GitHub Actions triggas automatiskt vid push till master-branchen och deployar hela appen till Azure.

6.1 Pusha koden till GitHub

Visual Studio → Git → Create Git Repository → GitHub → Privat repository:
AzureAppServiceAPI → Create and Push

6.2 Skapa workflow-filen

Skapade .github/workflows/deploy.yml i rotmappen av projektet (viktigt: inte i projektmappen utan på samma nivå som .sln-filen).

Workflow-filen innehåller:

- ☐ on: push branches: master – triggas vid push till master
- ☐ actions/checkout@v4 – hämtar koden
- ☐ actions/setup-dotnet@v4 med dotnet-version: '10.0.x'
- ☐ dotnet build --configuration Release
- ☐ dotnet publish AzureAppServiceAPI/AzureAppServiceAPI.csproj --configuration Release --output ./publish
- ☐ azure/webapps-deploy@v3 – deployar till Azure

6.3 Activate Basic Auth Publishing

Azure Portal → productapi-essan → Settings → Configuration → General settings → SCM Basic Auth Publishing Credentials: ON

6.4 GitHub Secret – Publish Profile

Azure Portal → productapi-essan → Overview → Download publish profile → Spara XML-filen

GitHub: Repository → Settings → Secrets and variables → Actions → New repository secret

Namn: AZURE_WEBAPP_PUBLISH_PROFILE

Value: Klistra in hela XML-innehållet från publish profile-filen

6.5 Deploy

git add . && git commit -m "Deploy" && git push origin master → GitHub Actions kör automatiskt

7. Application Insights – Loggning och övervakning

7.1 Skapa Application Insights

```
az monitor app-insights component create --app insights-essan-25 --location swedencentral  
--resource-group "RG-Essan-Alshakerchi-10d10a-DotNetCloudDeveloper-VT-Mars-Goteborg" --application-type web
```

7.2 Hämta Connection String och sätt App Setting

```
az monitor app-insights component show --app insights-essan-25 --resource-group "RG-Essan-..." --query "connectionString" -o tsv
```

Kopiera output och sätt det som App Setting:

```
az webapp config appsettings set --name productapi-essan --resource-group "RG-Essan-..." --settings  
APPLICATIONINSIGHTS_CONNECTION_STRING="<kopiera connection string här>"
```

7.3 Vad kan man övervaka?

- ☐ Live Metrics – realtidsdata om aktiva requests
- ☐ Failures – detaljerade felloggar med stack traces
- ☐ Performance – svarstider per endpoint
- ☐ Logs – Kusto Query Language (KQL) för avancerad analys

8. Säkerhet – HTTPS, IP-restriktion, Backup

8.1 Aktivera HTTPS

```
az webapp update --name productapi-essan --resource-group "RG-Essan-..." --https-only true
```

Med --https-only true omdirigeras alla HTTP-anrop automatiskt till HTTPS (HTTP 301 Redirect).

8.2 IP-restriktion

Först hitta din IP-adress:

```
curl https://api.ipify.org
```

Sedan sätt IP-restriktionen:

```
az webapp config access-restriction add --name productapi-essan --resource-group "RG-Essan-..." --rule-name "MinDator" --action Allow --ip-address 46.212.34.173/32 --priority 100
```

/32 betyder exakt en IP (ingen range). Alla andra IP-adresser nekas automatiskt.

8.3 Daglig backup

Backup konfigurerades via Azure Portal (CLI-syntaxen förändrades i nyare versioner):

Azure Portal → productapi-essan → Backups → Configure custom backups

- ☐ Storage account: stopproductapi1
- ☐ Container: app-backups
- ☐ Schedule: Every 1 Day
- ☐ Start time: 14:15:14

9. Autoskalning

Autoskalning justerar antal instanser baserat på CPU-belastning. S1-planen krävs.

9.1 Skapa autoscale-profil

```
az monitor autoscale create --resource-group "RG-Essan-..." --resource "ASP-ProductAPI" --resource-type "Microsoft.Web/serverfarms" --resource-namespace "Microsoft.Web" --name "autoscale-essan" --min-count 1 --max-count 3 --count 1
```

9.2 Regel – skala ut vid hög belastning

```
az monitor autoscale rule create --resource-group "RG-Essan-..." --autoscale-name "autoscale-essan" --scale out 1 --condition "CpuPercentage > 70 avg 5m"
```

Om CPU > 70% under 5 min läggs en instans till.

9.3 Regel – skala in vid låg belastning

```
az monitor autoscale rule create --resource-group "RG-Essan-..." --autoscale-name "autoscale-essan" --scale in 1 --condition "CpuPercentage < 30 avg 5m"
```

Om CPU < 30% under 5 min tas en instans bort. Min-instanser: 1, Max: 3.

10. Azure Storage Account

Storage Account lagrar statiska resurser och backuper. I en e-handelslösning skulle produktbilder lagras här.

10.1 Skapa Storage Account

```
az storage account create --name stopproductapi1 --resource-group "RG-Essan-..." --location swedencentral --sku Standard_LRS
```

Standard_LRS (Locally Redundant Storage) – billigaste alternativet, lagrar data i ett datacenter.

10.2 Skapa containers


```
az storage container create --name "app-files" --account-name stoproductapi1
az storage container create --name "app-backups" --account-name stoproductapi1
```

10.3 Containers

Container	Användning
app-files	Statiska filer – bilder, dokument
app-backups	Dagliga backuper av App Service

11. Verifikation – applikationen fungerar

Efter alla resurser skapats och appen deployats via GitHub Actions testades applikationen mot den publika URL:en i Postman.

Test-resultat

Test	Resultat
GET /api/Products	200 OK – tom lista
POST /api/Categories {"name":"Elektronik"}	200 OK
POST /api/Products {name, price, categoryId:1}	200 OK
GET /api/Products	200 OK – produkt med kategori

Alla endpoints returnerade 200 OK. Appen är live och fullt funktionell.

12. Källförteckning

Alla källor är från Microsoft Learn (officiell Microsoft-dokumentation).

- ASP.NET Core Web API – <https://learn.microsoft.com/en-us/aspnet/core/web-api/>
- Entity Framework Core – <https://learn.microsoft.com/en-us/ef/core/>
- Azure App Service – <https://learn.microsoft.com/en-us/azure/app-service/>
- Azure CLI – <https://learn.microsoft.com/en-us/cli/azure/>
- Azure SQL – <https://learn.microsoft.com/en-us/azure/azure-sql/>
- Key Vault – <https://learn.microsoft.com/en-us/azure/key-vault/>
- Managed Identity – <https://learn.microsoft.com/en-us/azure/app-service/overview-managed-identity>

- GitHub Actions – <https://learn.microsoft.com/en-us/azure/app-service/deploy-github-actions>
 - Application Insights – <https://learn.microsoft.com/en-us/azure/azure-monitor/app/app-insights-overview>
 - App Service Security – <https://learn.microsoft.com/en-us/azure/app-service/overview-security>
 - App Service Backups – <https://learn.microsoft.com/en-us/azure/app-service/manage-backup>
 - Autoscale – <https://learn.microsoft.com/en-us/azure/azure-monitor/autoscale/autoscale-overview>
 - Azure Storage – <https://learn.microsoft.com/en-us/azure/storage/blobs/storage-blobs-overview>
-